ChatGPT

Executive Summary

Chitra AI is envisioned as a **free-first, multi-tenant AI assistant platform** for small businesses. The MVP will use only free or open-source services (Supabase, Cloudflare, Render, etc.) with sensible quotas, deferring any paid tiers until absolutely needed. The core features include: conversational Q&A with business-specific knowledge (via RAG), appointment booking, lead capture, and omnichannel chat (website widget, WhatsApp/Messenger, etc.). The business model is freemium: a generous free tier (e.g. limited agents/month, messages/month, users, data size) with clear upgrade triggers (exceeding quotas or adding white-label features). A phased roadmap (V1–V4) will grow from a simple single-tenant chat bot to a robust multi-tenant SaaS with analytics, high security, and agency capabilities.

This report details the **architecture and development plan**: multi-tenant data design with RLS, user onboarding (signup, industry and knowledge loading via web crawl/file/drive sync), deployment options (JS widget, WordPress plugin, QR code link), integrations (WhatsApp, Instagram, Messenger via APIs), booking flow (tool calls and Cal.com integration), lead capture (email/SMS/webhook), human handoff, monitoring/analytics, and full tech stack. It also covers security (Cloudflare Turnstile, WAF, rate limits), RAG (chunking, embeddings, pgvector), embedding and LLM options (Groq API primary, OpenRouter fallback), API design (chat, knowledge, bookings), caching (Cloudflare KV/Upstash), storage (Cloudflare R2), auth (Supabase Auth), hosting (Render + Vercel/Cloudflare Pages), costs/free-tier limits, and migration paths.

Phases V1–V4 with milestones and effort estimates are provided, along with testing/QA steps and a launch checklist. Recommended tech stack and a draft database schema (tables with key fields) are in tabular form. Key sequences (onboarding, reservation, tool invocation) are illustrated with Mermaid diagrams. All technical choices emphasize free tiers and scalability.

1. Business Model

- **Freemium Tier**: Offer a *per-org free tier* with modest quotas. For example, each business (tenant) might get a free bot with up to *N* monthly conversations, *X* knowledge documents, and *Y* booking events per month. When limits are reached (e.g. active users, messages, or storage usage), prompt to upgrade or throttle gracefully.
- **Upgrade Triggers**: Exceeding quotas or requiring premium features (e.g. unlimited conversations, removal of “Chitra” branding, advanced analytics). Paid plans could unlock higher rate limits on LLM calls (Groq dev tier), priority support, White-Labeling (custom domain, branding removal), or agency features (manage multiple client accounts).
- **White-Label / Agency Offering**: In later phases (V3+), allow agencies to rebrand Chitra AI for clients, manage multiple client tenants under one organization, and provide service-level features. This may involve custom onboarding and enterprise features (e.g. dedicated support, single-sign-on).
- **No Upfront Payment**: Initially require *no credit card* to sign up. Groq’s free tier itself requires no card【6†L33-L41】, and we’ll mirror that policy. Optional upgrades to paid tiers (Groq Developer tier, Render paid instances, etc.) can come later.

2. User Onboarding Flow

An intuitive onboarding experience is crucial. Key steps:

1. **Signup / Account Creation**: User visits Chitra's site and signs up (email/password or OAuth).
2. **Business Profile**: Prompt the user to enter basic business info: name, address, industry/ category, time zone, etc.
3. **Select Chatbot Persona/Industry**: Offer industry templates (e.g. "Restaurant", "Salon", "Retail") to tailor the bot's initial system prompt and knowledge. The user selects the industry or service type.
4. **Teach the Bot Knowledge** (choose any combination):
5. **Website Crawl**: The user provides a URL (e.g. their company website). The system uses a crawler to extract text and generate embeddings (RAG data).
6. **File Upload**: Allow uploading documents (PDF, DOCX, text), images, or slides; these are chunked and vectorized.
7. **Manual Entry**: A simple text editor or form to paste FAQs or key details.
8. **Google Drive / Notion / Shopify Integration**: Provide OAuth connectors to import content (emails, docs, product info). For example, connect Google Drive to index Sheets/docs, or Shopify store to index product Q&A.
9. **Waiting / Processing**: Show progress as Chitra ingests data (embedding, indexing). This happens asynchronously (background jobs).
10. **Dashboard / Chat Test**: Once done, show a dashboard with a "Test Chat" interface so user can try the bot. Also display usage stats (remaining free quota).
11. **Install Bot**: Provide easy install options (JS snippet, WordPress plugin, QR code). See Section 3 for details.

This flow can be implemented with Next.js (React) pages or similar, backed by Supabase authentication. We will use **Supabase Auth** for user accounts, which integrates with Postgres and provides built-in RLS support on `auth.uid()` 【21†L73-L82】. On signup, create a new organization/tenant record.

```
flowchart LR
  A[Visitor] --> B{Has Account?}
  B -- No --> C[Signup Form]
  B -- Yes --> D[Login]
  C --> E[Create Account]
  E --> F[Business Profile]
  D --> F
  F --> G[Choose Industry/Template]
  G --> H[Load Knowledge: Crawl / Upload / API]
  H --> I[Bot Knowledge Indexing (RAG)]
  I --> J[Dashboard: Bot Ready]
  J --> K[Get Install Code (Widget/QR)]
```

Figure: Onboarding flow (user signup, business info, knowledge ingestion, bot ready).

3. Bot Installation Options

Once the bot is configured, the user needs to add it to their channels:

- **JS Widget**: Provide a small JavaScript snippet (like Intercom) that embeds the chat widget on any website. This widget will load our chat UI and communicate with Chitra's API.
- **One-Click Integration**: For popular platforms, offer a quick-install:
- **WordPress Plugin**: A simple WP plugin where user enters a Chitra API key or account token and the chat widget appears on their site.
- **QR Code / NFC (ChitraTap)**: Allow generating a QR code or NFC tag that links to a standalone chat page for the business, or instantly launches the bot in mobile. Useful if the business wants customers to scan and chat via mobile.
- **Direct Link**: Each bot has a unique URL (e.g. `chitra.ai/bot/abc123`) that can be shared. This link opens a web chat interface (hosted on Cloudflare Pages/Vercel).
- **Messaging Integrations**: See next section for WhatsApp/Instagram/Messenger.

All these frontends call our backend Chat API via REST/WebSockets.

4. Messaging Platform Integration

To reach customers where they already chat, integrate Chitra with social/chat platforms:

- **WhatsApp / Instagram (Meta) / Messenger**: Use official APIs or middleware:
- *WhatsApp Business API*: Could use Twilio's API for WhatsApp to receive/send messages【44†L917-L924】. Twilio offers a free trial including some messages. Alternatively, use Meta's own API (via Facebook's Graph API) for WhatsApp/Instagram/FB Messenger. We'll abstract message sending so it works similarly to web chat.
- *Instagram DM API*: (Part of Meta Graph API) to send/receive messages.
- *Facebook Messenger API*: Use the Messenger Platform.
- On receiving a message, the bot runs the same conversation flow (calls our LLM/chat API) and replies via the platform's API.
- **Fallback / Notification**: If official APIs are too complex now, a simplified approach is to provide a "Chat on WhatsApp" link (wa.me) for business, but true two-way requires API integration.

No official free unlimited integration exists, but we plan to start with basic webchat and add WhatsApp/Messenger once MVP is stable.

5. Reservation & Booking Flow

Chitra will offer booking functionality (making appointments/reservations) via tool calls and calendar sync:

1. **User Request**: Customer asks something like "Book me a table on Friday at 7pm" in chat.
2. **Intent + Tool Call**: The LLM identifies intent and calls a `check_availability` tool (schema defined as function). This is implemented via our tool system.
3. **Availability Check**: The `check_availability` function queries the business's calendar (either internal or via Cal.com API).
4. **Response to User**: Chitra (LLM) sees the tool result and asks for customer details (name, phone, etc.).

5. **Booking Action**: The user provides details; LLM calls another tool `create_booking` with those details.
6. **Calendar Event**: `create_booking` uses Cal.com API (free individual plan) to schedule the slot on the business's calendar (Cal.com free allows unlimited event types and bookings【11†L90-L98】).
7. **Confirmation**: The LLM then confirms to the user: e.g. "Your reservation #CH1234 is confirmed."
8. **Owner Notification**: Simultaneously, we notify the business owner (via email/SMS/WhatsApp) of the new reservation.

By using Cal.com's robust API (free tier includes unlimited bookings【11†L90-L98】) we avoid building scheduling from scratch. Cal.com provides webhook callbacks and a React component library (Cal Atoms) for embedding if needed【13†L201-L210】. The reservation flow uses two steps of function-calling (check then create), managed by our backend.

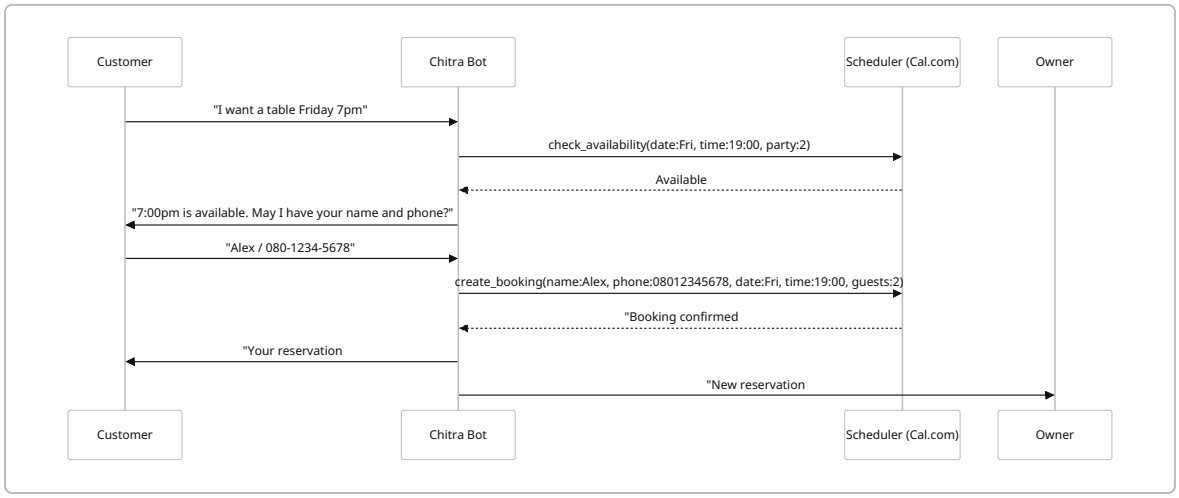


Figure: Booking/reservation sequence with tool calls (check availability, create booking) and notifications.

6. Lead Capture & Notifications

- **Lead Capture**: Whenever the bot collects a new customer's contact (name, email/phone), we store it as a lead in the database. A `create_lead(name, contact)` tool can be invoked by the LLM at any time. These leads are also sent via configured channels (webhook or email).
- **Notifications**:
- **Email/SMS/WhatsApp**: When certain events occur (new user sign-up, reservation, incoming lead), send a notification to the business. This can be done via integration services:
    - *Email*: Use an SMTP service or an API (e.g. SendGrid, Mailgun). Cloudflare Workers KV/ SendGrid.
    - *SMS/WhatsApp*: Use Twilio or an SMS API (free trials exist).
    - *Webhook*: Allow advanced users to provide a webhook URL to receive JSON events.
- Notifications ensure the owner can follow up promptly.
- **Human Handoff**: If the bot cannot handle a query (e.g. "talk to a human"), we can queue the conversation for a staff member. This could be a simple forwarding of the chat transcript to an email or posting to Slack. (No paid integration needed initially; an email to owner can serve as "handoff".)

7. Analytics & Usage Tracking

- **Built-in Analytics**: Track usage metrics (conversations count, active users, bookings made) per tenant.
- **Event Tracking**: Use **PostHog** (self-hosted or cloud) for detailed analytics (page views, feature usage). PostHog's free tier allows up to **1M events/month**【38†L136-L144】, which should suffice for early usage. Its dashboard shows active users, sessions, funnels, etc.
- **Dashboard**: In the tenant's admin area, show key stats (weekly conversations, bookings, leads, etc.) and any errors or rate-limit warnings.
- **Logging & Errors**: Use Supabase Logs or an external service (Sentry) to record backend errors.

By instrumenting the front-end (React) and backend, we can monitor adoption and identify any bottlenecks.

8. Multi-Tenant Data Model & RLS

Every customer (tenant) must see only their own data (knowledge, chats, bookings, leads). We will use a single Postgres database with **Row-Level Security (RLS)** to enforce isolation【21†L74-L82】. Key design:

- **Tenants/Organizations**: Table `organizations(id, name, owner_user_id, ...)`.
- **Users**: Supabase Auth users (each user belongs to one org). All data tables include a `organization_id` FK.
- **Knowledge Documents/Chunks**: Tables like `documents`, `document_sections` (as in Supabase example). Each row has `organization_id`.
- **RLS Policies**: Define policies such that any `SELECT` (vector similarity, etc.) is automatically filtered by `current_setting('jwt.claims.org_id')` (Supabase passes the user's org_id)【21†L74-L82】【21†L239-L242】. For example, `WHERE organization_id = auth.uid_org()`.
- **Vector Embeddings**: We store embeddings in pgvector columns (e.g. `vector(1536)`) on `document_sections`. Supabase's RLS applies to these too【21†L74-L82】, ensuring a tenant only sees their chunks. Buildoto's guide emphasizes that RLS can secure vector searches by filtering tenant metadata【4†L16-L21】【4†L48-L53】.
- **Isolation**: No single "vault" table for all tenants; each row belongs to one tenant. This prevents cross-tenant leaks.

Sample tables (keys only):

Table	Key Fields
organizations	id, name
users	id, email, hashed_password, organization_id
documents	id, organization_id, title, url, created_at
document_sections	id, document_id (FK), organization_id, content, embedding (vector)
chat_history	id, organization_id, user_id, message, response, timestamp
bookings	id, organization_id, customer_name, contact, datetime, details

Table	Key Fields
leads	id, organization_id, name, contact, source, created_at

(All tables include `organization_id` *and RLS policies to enforce isolation.)*

9. Security, Rate Limiting & Abuse Protection

We must protect both the platform and tenants from abuse:

- **Captcha/Turnstile**: Use **Cloudflare Turnstile** on public forms (signup, content upload) to block bots. Turnstile is a free captcha alternative that doesn't interrupt users【36†L0-L4】.
- **Cloudflare WAF**: Deploy Cloudflare's Web Application Firewall (WAF) rules to block SQL injections, XSS, common web attacks.
- **Rate Limiting**: Use Cloudflare or our API gateway to limit requests per IP or per token (e.g. 1000 requests/hour per tenant). This prevents excessive usage under the free plan.
- **Auth Protection**: Use Supabase Auth with JWTs. All API calls require a valid JWT, which includes the `organization_id` claim for RLS.
- **Network Limits**: On Render's free plan, egress is limited; avoid heavy file uploads/downloads. (Caution: free services may spin down – see Render docs【42†L110-L119】).
- **Data Privacy**: Ensure bot responses do not leak private info. Supabase's data policies help (RLS). We won't log PII unnecessarily.

No user data from one tenant is accessible to another, per the RLS approach. We will also enforce **rate quotas per tenant** at the application level (e.g. max messages/day under free plan).

10. Retrieval-Augmented Generation (RAG) Architecture

Chitra's knowledge answering uses RAG: indexing user-provided content (web pages, docs, etc.) as embeddings and retrieving similar chunks during chat. Key points:

- **Data Chunking**: Break long docs into 300–500 token chunks (with some overlap) for vector indexing.
- **Embeddings**: Compute text embeddings for each chunk and store in `document_sections.embedding`. Options:
- **Local/Open Models**: Use an open-source embedding model like sentence-transformers "all-MiniLM-L6-v2" or E5-small. These run inference free (server load only), avoiding API costs.
- **API Embeddings**: As fallback, could use OpenAI or Groq embeddings via OpenRouter if free limit allows, but since we target *no payments*, local embeddings are preferred.
- **Vector DB**: Use **Supabase/Postgres with pgvector** (no extra cost on Pro or above; free tier supports vectors) or Cloudflare D1 (SQLite) with vector extension. Supabase integration is simpler since we already have RLS. Pgvector indexing and similarity search will use cosine/inner-product queries.
- **RLS on Vectors**: Supabase applies RLS to vector tables (as in [21]), so a similarity search query like `SELECT ..., vector <-> query_embedding as sim FROM document_sections` automatically filters to the tenant's rows【21†L239-L242】.
- **Similarity Search**: During chat, form a retrieval query over embeddings (with tenant constraint). Retrieve top-k chunks (say k=5) that match the user's question.
- **Prompt Construction**: Combine retrieved chunks and relevant context into the LLM prompt (system + user + retrieved docs) to answer questions.

- **Chunk Storage**: Store original text for each chunk (for source references or citations).

Architecture style is similar to AWS Bedrock multi-tenant RAG: one global vector store with tenant filtering【4†L16-L21】. We avoid per-tenant separate DBs by RLS policy. This meets best practices: "push isolation into the database" rather than code【4†L16-L21】.

11. Embedding & Model Options

- **Primary LLM**: **Groq API** (GPT-compatible). Groq offers a free tier with generous limits (e.g. 14,400 daily requests with their 8B model【6†L72-L80】) and compatibility with OpenAI's API. We will use Groq for all chat/completions by default. If rate-limited or out of free credits, **OpenRouter** can be configured as a fallback (aggregating other models; though it uses paid credits).
- **Embedding Models**: For RAG we can use:
- *Local* ("on-prem"): e.g. Hugging Face Transformers (sentence-transformers) running on our backend (no token costs). E.g. `intfloat/multilingual-e5-small-v2` (free) or `all-MiniLM-L6-v2`. The tradeoff is CPU load vs no API cost.
- *OpenAPI-compatible*: If needed, we could route embedding calls through OpenRouter's API (which passes through to real APIs with no markup【17†L183-L192】), but this still costs credits.
- **Provider Abstraction**: Implement a thin layer so that model calls (chat or embeddings) go through an abstraction. First choice is Groq; if Groq is down or quota exceeded, route to OpenRouter (which in turn may use OpenAI or other models). This ensures uptime (OpenRouter falls back automatically).
- **Tool Calling / Structured Output**: We define JSON schemas for each tool (function) the LLM can invoke. Groq's **Tool Use** feature supports structured JSON function calls【19†L228-L236】. In practice, we send a POST to Groq with `tools: [ {name, description, parameters-schema}, ... ]` and the conversation messages. Groq will output a `"tool_calls": [...]` array【19†L280-L288】 if it decides to use a tool. Our backend parses this, executes the function (e.g. `check_availability`), then feeds the result back to the model for a final response.

12. Tools & Function Schemas

We will predefine a set of JSON tools (similar to OpenAI's function-calling). Examples:

- `check_availability({ "date": "YYYY-MM-DD", "time": "HH:MM", "party_size": int })`: Checks calendar for a free slot.
- `create_booking({ "name": string, "contact": string, "date": "...", "time": "...", "details": string })`: Books the slot (writes to DB/Cal.com).
- `search_documents({ "query": string })`: Returns top knowledge chunks (e.g. for direct API retrieval).
- `create_lead({ "name": string, "contact": string, "source": string })`: Saves a lead and triggers notifications.

Each tool has a name, description (for the model), and a JSON Schema for parameters【19†L226-L236】. The LLM outputs calls like `"function": {"name": "check_availability", "arguments": "{\"date\": \"2026-08-30\", \"time\":\"19:00\", \"party_size\":2}"}` 【19†L280-L288】. We then perform the action in our code and continue the conversation.

13. API Design

We will expose a RESTful (or GraphQL) backend API on Render for:

- **Chat Endpoint** (`POST /api/chat`): Accepts user message + session ID + optional context. Backend fetches any needed context (history, RAG) and calls Groq with tools. Responds with model's answer (or intermediate JSON if function call, but our UI abstracts that).
- **Knowledge Endpoint** (`POST /api/knowledge`): For uploading documents, URLs, syncing content. Triggers the ingestion pipeline (chunking + embedding). May return a job ID.
- **Bookings Endpoint** (`GET/POST /api/bookings`): To query or create bookings directly (back-end to Cal.com).
- **Leads Endpoint** (`GET /api/leads`): To list leads captured.
- **Auth Endpoints**: Handled by Supabase (via JWT).
- **Frontend Integration Endpoint** (`GET /embed.js`): Serves the chat widget JS.

Each API call checks the Supabase JWT to get `organization_id` and enforces RLS policies.

We'll likely use Next.js API routes or a lightweight Node/Python server on Render (free tier supports Node/Docker).

14. Caching & Queues

- **Cache**: Use Cloudflare **Workers KV** (free tier has 100K reads/day) for caching LLM responses or static assets. Also consider Upstash Redis (free tier available) for ephemeral caching of recent queries to improve performance.
- **Background Jobs**: For long-running tasks (content ingestion, periodic tasks), use a job queue:
- Render offers **Background Workers** or one can self-manage with BullMQ/RabbitMQ (hosted on Render or Upstash/RabbitMQ Cloud). For example, after web crawl, queue a job to embed chunks.

- Use **Supabase Edge Functions** (if needed) for lightweight tasks triggered by DB changes.

- **Serverless Cron**: If periodic tasks are needed (e.g. nightly data refresh), we could use Render's Cron Jobs or Cloudflare Cron Triggers.

15. Storage

- **Vector and Relational Data**: Postgres on Supabase (free 500 MB) stores structured data, embeddings (pgvector), user sessions, etc.
- **Object Storage**: **Cloudflare R2** (free 10 GB storage/month【35†L183-L192】) for storing uploaded files (images, videos, large documents). Objects in R2 can be fetched directly by Workers or our backend. R2 also has generous free Class A/B ops (1M/10M) to serve small volumes of traffic【35†L183-L192】.
- **File Uploads**: When user uploads docs/images, store them in R2 and reference their URL in the database.

16. DevOps & Hosting

- **Backend (API)**: Host on **Render** (free tier). We get 750 free instance hours/month【42†L112-L119】 (enough for one service ~24/7). A free Postgres (1GB, 30-day TTL)【42†L201-L209】 can

be used for dev but must plan to upgrade to paid Pro for production since data expires in 30d. Render's free PostgreSQL is limited (1GB)【42†L201-L209】, so initial data must be lightweight. (Alternatively, host Postgres on Supabase with a free project.)

- **Frontend**: Host on **Vercel** or **Cloudflare Pages** (free tier, auto SSL). A static Next.js/React app for the dashboard and widget.
- **Edge Workers**: Cloudflare Workers (free plan) can be used optionally for caching or simple dynamic routing of the chat widget, but not mandatory. Workers free plan allows 100K KV reads/day【34†L810-L818】.
- **Database**: Primary DB on Supabase (Postgres, 500MB free【47†L26-L34】). We use Supabase both for RLS and as the official storage. (Alternatively, Render Postgres in free mode for initial dev, then migrate to Supabase for RLS and scaling.)
- **Monitoring**: Deploy PostHog on a small cloud instance (free 1M events)【38†L136-L144】, or use PostHog Cloud free tier. Use Supabase's logs and/or Sentry for error reporting.
- **CI/CD**: Use GitHub Actions to deploy to Render and Vercel on push.
- **Domain**: Map custom domain to frontend (e.g. `app.chitra.ai`) via Cloudflare DNS. Use SSL by default.
- **Secret Management**: Store API keys (Groq, OpenRouter, Cal.com, Twilio) in Render secret env vars.

17. Costs, Free-Tier Limits & Migration Paths

Service	Free Tier (Limit)	Migration Path (Paid)
Groq AI	~50 requests/min (8K tokens/min), 14,400 req/day on gpt-oss-8b【6†L72-L80】; no CC needed【6†L33-L41】	Upgrade to Groq Developer (pay per token)【9†L219-L228】
OpenRouter	Requires buying credits (no free inference) – used only as fallback	Use pay-as-you-go, or direct provider APIs
Supabase	500 MB Postgres, 50k MAU, 1GB storage【47†L26-L34】; unlimited API calls	Pro plan ($25+) for more space, compute credits
Cloudflare R2	10 GB storage, 1M Class-A, 10M Class-B ops/month【35†L183-L192】	Pay per usage, or use S3/Azure Blob when scale
Cloudflare KV	100k reads/day, 1k writes/day, 1GB KV storage【34†L810-L818】	Workers paid plan ($5) for higher limits
Render (backend)	750 instance-hours/month【42†L112-L119】, Free Postgres 1GB (30d expiry)【42†L201-L209】	Move to paid instance ($7+/month) to avoid sleep and data expiry
Cal.com	Unlimited bookings (free forever)【11†L90-L98】, REST API access	Team/Org plan ($12+/user/mo) for API, multi-user features
PostHog	Free 1M events/month, 5K replays, etc.【38†L136-L144】	Usage-based beyond free (close to $0.00005/event)
Twilio/ WhatsApp	Trial messages/free templates limited, no card needed for Sandbox	Use Twilio pay-as-you-go (first 1K free for engagement suite【44†L913-L920】), or other SMS API

Service	Free Tier (Limit)	Migration Path (Paid)
Upstash (Redis)	Free 20k ops, 128MB, 200ms retention etc (varies)	Paid Redis, or cache in MemoryStore if needed

(Free tier data as of 2026; costs subject to change. The migration path is to upgrade to paid plans or alternative providers when usage demands.)

18. Tech Stack & Libraries

Frontend: React (Next.js) + TypeScript, Tailwind CSS or Chakra UI, supabase-js SDK, mermaid.js (for docs/visualization). Widget uses simple HTML/JS/CSS.

Backend: Node.js with Express or Fastify (or Next.js API routes) in TypeScript. Use **Supabase JS** for DB and Auth, or Prisma. Hosted on Render.

LLM & AI:

- **Groq SDK** (openai-compatible) for chat completions and structured outputs【15†L131-L134】.
- **OpenRouter SDK** (for fallback).
- **Vector/Embeddings:** `pgvector` extension on Postgres, and either HuggingFace Inference (`@huggingface/inference` or `transformers` in Python/Node) for embeddings.
- **RAG:** Might use `llama_index` or direct SQL for similarity search.

Database: Supabase (Postgres + pgvector), with RLS enabled.

Storage & Caching: Cloudflare R2 (object storage) via official SDK or AWS S3 SDK; Cloudflare Workers KV (cache) via Cloudflare APIs; optionally Upstash Redis for session cache.

Auth: Supabase Auth (email/password), JWTs, role-based (RLS). Use Supabase UI or React-AuthKit.

CI/CD: GitHub Actions; render.com auto-deploy; Vercel auto-deploy.

Monitoring: PostHog SDK in frontend/backend; Sentry or Upstash logs for errors.

Sequence Diagrams / Flowcharts: Use Mermaid (live in docs). For example, the above flows use Mermaid syntax.

Below is the **database schema outline** (tables with key columns):

Table	Key Fields (PK, FKs)
organizations	`id (PK)`, `name`, `owner_user_id (FK users.id)`, …
users	`id (PK)`, `email`, `hashed_password`, `organization_id (FK orgs.id)`, … (managed by Supabase Auth)
documents	`id (PK)`, `organization_id (FK)`, `title`, `url`, `created_at`
document_sections	`id (PK)`, `document_id (FK documents.id)`, `organization_id (FK)`, `content (text)`, `embedding (vector)`

Table	Key Fields (PK, FKs)
chat_history	`id (PK)`, `organization_id (FK)`, `user_id (FK)`, `role ('user'/'assistant')`, `message`, `created_at`
bookings	`id (PK)`, `organization_id (FK)`, `customer_name`, `contact_info`, `datetime`, `booking_reference`
leads	`id (PK)`, `organization_id (FK)`, `name`, `contact (email/phone)`, `source`, `created_at`
settings	`organization_id (PK)`, `time_zone`, `notification_emails`, `whatsapp_number`, …
api_keys (optional)	`id (PK)`, `organization_id (FK)`, `key_hash`, `created_at`

All tables use an `organization_id` column and we enable Postgres **RLS** so that queries automatically filter by the current user's org【21†L74-L82】.

19. Phased Roadmap (V1–V4)

V1 (Core MVP, 1–2 months):

- User signup/login, organization creation (Supabase Auth).
- Basic chat UI + backend calling Groq.
- Single-tenant knowledge ingestion (website crawl and manual text); vector store in Postgres.
- Simple Q&A using RAG.
- Basic reservation flow (tools to check/create bookings with Cal.com).
- Test messaging via web widget.
  Deliverables: Working prototype with one tenant, simple UI, 5–10 test businesses.
  Effort: 4–6 weeks by small team (or 8–12 wks solo).

V2 (Multi-Tenant & Integrations, 2–3 months):

- Enable multi-tenant RLS (each org separate)【21†L74-L82】.
- Onboarding flows (industry selection, drive/Notion sync, file uploads).
- Install options: JS widget, sample WordPress plugin, QR code generator.
- WhatsApp/Messenger integration (initial POC via Twilio or Facebook API).
- Lead capture (save leads, email notifications).
- Analytics integration (PostHog tracking).
  Deliverables: Tenant dashboard, multi-org support, widget embed, basic analytics.
  Effort: +6–8 weeks.

V3 (Stabilize & Extend, 2–3 months):

- Polish NLP and prompts; add fallback provider (OpenRouter) logic.
- Human handoff (chat transfer to email/Slack).
- Enhance UI (Dashboard stats, usage tracking, logs).
- Performance improvements (caching, async jobs).
- UX flows (error handling, email verification, reset).

- Security (Cloudflare Turnstile on forms, API rate limits).
  Deliverables: Beta product with stable multi-tenant ops, ready for limited launch.
  Effort: +6–8 weeks.

V4 (Scaling & White-Label, 3+ months):

- White-labeling features (custom domain, remove branding).
- Agency dashboard (manage multiple orgs).
- Mobile app or advanced widget customization.
- SLA/monitoring (uptime stats, backups).
- Launch marketing readiness (pricing page, docs, tutorial).
  Deliverables: Full SaaS platform, ready for public launch and partners.
  Effort: +10–12 weeks.

(Timelines assume a small dev team; solo development will roughly double these efforts.)

20. Testing & QA

- **Unit Tests:** For backend functions (tool handlers, API endpoints). Use Jest or PyTest.
- **Integration Tests:** Simulate flows (signup → ingest → chat) end-to-end.
- **Load Testing:** Especially the LLM calls (simulate N users chatting to ensure rate limits and caching work).
- **Security Testing:** Verify RLS policies block cross-org access (try to query other org's data). Test Turnstile/Captcha.
- **Usability Testing:** Ensure the onboarding and widget UI is clear.
- **Regression:** Run automated tests on each commit (CI pipeline).

All critical features (sign-up, chat, booking) must have test coverage ~80%.

21. Launch Checklist

- [ ] Core features implemented & tested (chat, RAG, booking).
- [ ] Multi-tenancy secured (RLS policies verified)【21†L74-L82】【21†L239-L242】.
- [ ] Free quotas are configured and enforced (throttling alerts).
- [ ] All integrations configured (Groq API key, Cal.com keys, email/SMS service keys, etc.)
- [ ] Frontend deployed (SSL, domain).
- [ ] Analytics (PostHog) and error monitoring set up.
- [ ] Privacy policy / Terms updated (mention cloud data use, etc.).
- [ ] Documentation: user guide for onboarding + API docs.
- [ ] Backup plan: If Render Postgres (free) expires, have migration ready (e.g. to Pro DB).
- [ ] Payment mechanism (if any) or at least pricing page drafted.
- [ ] Test in staging with 3rd-party users.

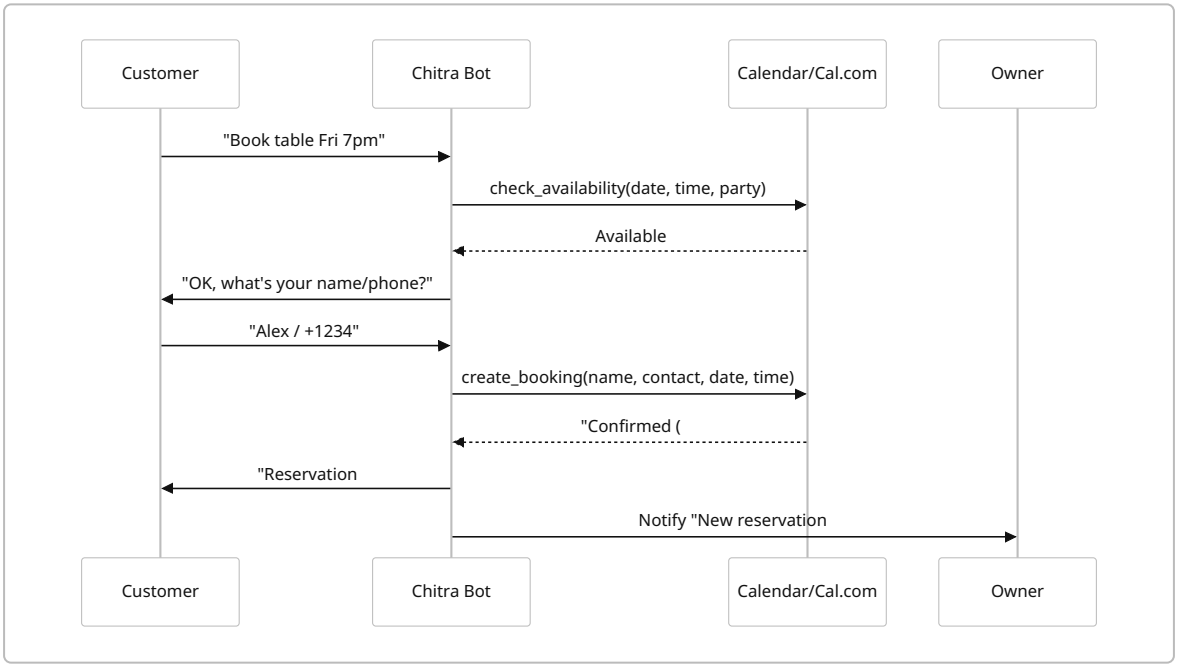
22. Sequence Diagrams

Below are high-level Mermaid diagrams for key flows. (In the actual product docs these can be rendered to images.)

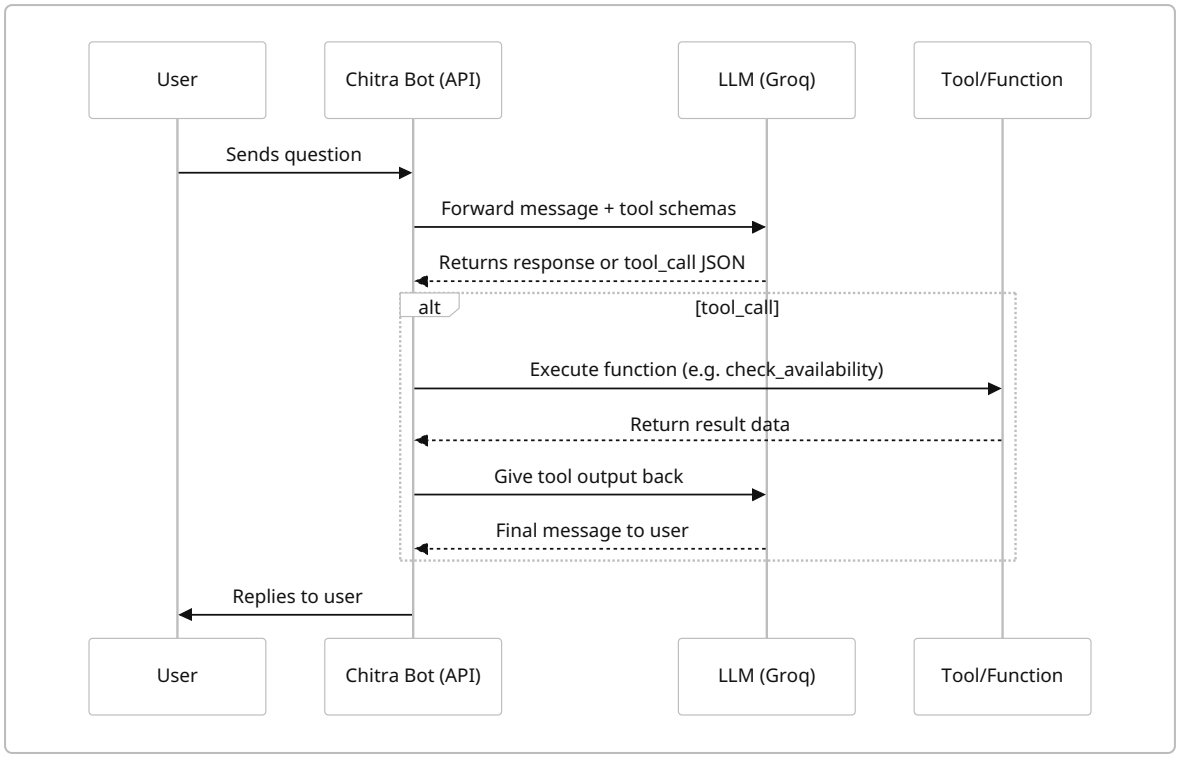
```
flowchart LR
    A[Visitor] --> B{Has account?}
```

```
B -- No --> C[Signup form]
B -- Yes --> D[Login]
C --> E[Create account]
E --> F[Business info form]
D --> F
F --> G[Choose industry/template]
G --> H[Add knowledge: Crawl/Upload/Sync]
H --> I[Bot training (RAG indexing)]
I --> J[Dashboard & widget code]
```

Onboarding flow: signup → business info → knowledge upload → bot ready.



Reservation booking flow using tool calls and notifications.



Chatbot message lifecycle: user query → LLM possibly tool-call → tool executes → back to LLM → respond.

23. Database Schema (Detail)

Below is a recommended schema outline (simplified). RLS policies enforce that `organization_id` equals current tenant's ID.

Table	Columns (type, notes)
organizations	`id (UUID PK)`, `name TEXT`, `owner_user_id UUID`, `created_at`
users	`id (UUID PK)`, `organization_id FK`, `email TEXT`, `password_hash`, `role`, …
settings	`organization_id (PK FK)`, `timezone`, `notify_email`, `whatsapp_number`, …
documents	`id (bigint PK)`, `organization_id FK`, `title TEXT`, `url TEXT`, `created_at TIMESTAMP`
document_sections	`id (bigint PK)`, `document_id FK`, `organization_id FK`, `content TEXT`, `embedding VECTOR`
chat_history	`id (bigint PK)`, `organization_id FK`, `user_id FK`, `role TEXT`, `message TEXT`, `timestamp TIMESTAMP`
bookings	`id (bigint PK)`, `organization_id FK`, `customer_name`, `contact_info`, `datetime TIMESTAMP`, `reference TEXT`, `status TEXT`

Table	Columns (type, notes)
leads	`id (bigint PK)`, `organization_id FK`, `lead_name TEXT`, `contact_info TEXT`, `source TEXT`, `created_at`
api_keys	`id (serial PK)`, `organization_id FK`, `key_hash TEXT`, `created_at`

(FK = foreign key referencing the owner org/user. Each table should have an RLS policy like `CHECK (organization_id = current_setting('jwt.claims.org_id')::uuid)`*, as in Supabase RLS examples【21†L74-L82】【21†L239-L242】.)*

24. Testing & QA Strategy

- **Unit Tests:** Test individual functions (knowledge ingestion, tool functions, API endpoints).
- **Integration Tests:** Simulate full chatbot sessions (e.g. a booking conversation) and verify correct DB writes, calendar updates.
- **RLS Testing:** Try queries as different tenants to ensure isolation (e.g. user A cannot see org B's docs).
- **Load/Stress Testing:** Mock many simultaneous users to ensure our free-tier limits hold (e.g. 750h on Render, 100k KV reads, Groq rate limits).
- **Security Testing:** Validate CAPTCHA works, ensure SQL injection is impossible (RLS helps), rate limits block excessive calls.
- **User Acceptance:** Beta test with a few friendly businesses to get feedback on flows and UI.

25. Launch Checklist

- [ ] Finalize and deploy Supabase schema + RLS policies.
- [ ] Deploy backend (Render) and frontend (Vercel/Cloudflare Pages).
- [ ] Configure Groq API key (free tier) and Cal.com API key in env.
- [ ] Set up any necessary webhooks (Cal.com, PostHog).
- [ ] Ensure Cloudflare Turnstile is on signup and upload endpoints.
- [ ] Verify Worker KV or caching is functional.
- [ ] Populate initial FAQ/knowledge templates for industries.
- [ ] Create demo tenant for public testing.
- [ ] Prepare support docs (FAQ, troubleshooting).

Sources: We relied on official documentation wherever possible. For example, Groq's docs and blog show its generous free tier【6†L72-L80】【9†L219-L228】; Supabase guides discuss RLS for vectors【21†L74-L82】【21†L239-L242】; Cal.com's pricing/guide highlights its free unlimited bookings and API-first design【11†L90-L98】【13†L201-L210】; Cloudflare's docs detail free R2/KV quotas【35†L183-L192】【34†L810-L818】; PostHog's site confirms a 1M-event free tier【38†L136-L144】. These guided our choices of free services and quotas.
